



Morpho - VaultV2 + MorphoVaultV1 Adapter supplying into an arbitrary ERC4626 Security Review

Auditors

Alireza Arjmand, Lead Security Researcher

Saw-mon and Natalie, Lead Security Researcher

Om Parikh, Security Researcher

Report prepared by: Lucas Goiriz

September 21, 2026

Contents

1	About Spearbit	2
2	Introduction	2
3	Risk classification	2
3.1	Impact	2
3.2	Likelihood	2
3.3	Action required for severity levels	2
4	Executive Summary	3
4.1	Scope	3
5	Findings	4
5.1	Medium Risk	4
5.1.1	The adapter does not revert on allocate if the <code>newAllocation</code> is <code>0</code>	4
5.1.2	Slippage protection is missing from adapter <code>deallocate</code> (and <code>allocate</code> endpoint)	4
5.1.3	Checks missing to make sure the exact number of assets withdrawn from deallocation	5
5.1.4	Perform atomic approvals and add checks when depositing assets into the yield source	5
5.2	Low Risk	5
5.2.1	Unsafe type casts might not be safe for a general <code>ERC4626</code> Vault	5
5.2.2	Temporary DoS of a <code>VaultV2</code> by a general <code>ERC4626</code> yield source	6
5.2.3	Faulty <code>ERC4626</code> yield sources can gradually drain idle or adapter-allocated liquidity	6
5.2.4	Skim can't be used to withdraw rewards which require authentication	7
5.2.5	Indirect shares donation to adapter may drift accounting	7
5.3	Informational	8
5.3.1	<code>VaultV2</code> allocation to a general <code>ERC-4626</code> via <code>MorphoVaultV1Adapter</code>	8
5.3.2	Potential discrepancies between <code>previewRedeem</code> , <code>redeem</code> and <code>withdraw</code>	10

1 About Spearbit

Spearbit is a decentralized network of expert security engineers offering reviews and other security related services to Web3 projects with the goal of creating a stronger ecosystem. Our network has experience on every part of the blockchain technology stack, including but not limited to protocol design, smart contracts and the Solidity compiler. Spearbit brings in untapped security talent by enabling expert freelance auditors seeking flexibility to work on interesting projects together.

Learn more about us at spearbit.com

2 Introduction

Morpho is a trustless and efficient lending primitive with permissionless market creation.

Disclaimer: This security review does not guarantee against a hack. It is a snapshot in time of Morpho - VaultV2 + MorphoVaultV1Adapter supplying into an arbitrary ERC4626 according to the specific commit. Any modifications to the code will require a new security review.

3 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

3.1 Impact

- High - leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium - global losses <10% or losses to only a subset of users, but still unacceptable.
- Low - losses will be annoying but bearable--applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

3.2 Likelihood

- High - almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium - only conditionally possible or incentivized, but still relatively likely
- Low - requires stars to align, or little-to-no incentive

3.3 Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

4 Executive Summary

Over the course of 2 days in total, [Morpho](#) engaged with [Spearbit](#) to review the [vault-v2](#) protocol. In this period of time a total of **11** issues were found.

Summary

Project Name	Morpho
Repository	vault-v2
Commit	425f6b1f
Type of Project	DeFi, AMM
Audit Timeline	Jan 7th to Jan 9th

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	4	0	4
Low Risk	5	0	5
Gas Optimizations	0	0	0
Informational	2	0	2
Total	11	0	11

4.1 Scope

The security review had the following components in scope for [vault-v2](#) on commit hash [425f6b1f](#):

```
src
├── VaultV2.sol
├── adapters
│   └── MorphoVaultV1Adapter.sol
```

This report covers a general case where a parent `VaultV2` supplies into an `ERC-4626` vault through `Morpho VaultV1Adapter`. The goal is to define the conditions under which this integration is safe and the behaviors or vault types for which it is unsuitable. The elements mentioned should be used as a guideline for developers and security researchers, not as a sufficient checklist to make such an integration safe.

5 Findings

5.1 Medium Risk

5.1.1 The adapter does not revert on allocate if the `newAllocation` is `0`

Severity: Medium Risk.

Context: [MorphoVaultV1Adapter.sol#L73](#).

Description: The adapter does not revert on allocate if (the checks for the scenarios below are lacking).

- The `newAllocation` is `0` or.
- No new shares are minted for the adapter.

This could happen for example if there was an inflation attack.

Recommendation: `allocate(...)` should revert when:

- `newAllocation` returned is `0` or.
- `IERC4626(V).balanceOf(address(this))` remains unchanged when compared before and after calling `IERC4626(V).deposit(assets, address(this))`.

Morpho: The recommendation is a bit too strict. `vault-v1` can in fact mint no shares on `allocate(1)` and not increase the allocation. What is required is that the per operation loss is "contained" with regard to the gas required to make an allocation.

5.1.2 Slippage protection is missing from adapter `deallocate` (and `allocate` endpoint)

Severity: Medium Risk.

Context: [MorphoVaultV1Adapter.sol#L71](#), [MorphoVaultV1Adapter.sol#L82-L89](#).

Description: In adapter `deallocate` one calls `IERC4626(V).withdraw(assets, ...)` without any slippage protection on the amount of shares burnt to withdraw the provided `assets`.

Recommendation: One can use the unused provided input parameters to `deallocate` to perform a slippage protection check. The maximum amount of shares to be burnt during the withdraw can be encoded in the `data` parameter.

This protection can be selectively added to the original endpoints called on the parent vault by inspecting the 3d input parameter `bytes4`. This can come from:

- `deallocate`
- `exit`
- `forceDeallocate`

The same slippage protection can be added to the `allocate` endpoint and the encoded share would be a minimum value of shares minted for a given amount of `assets`.

Morpho: The best solution is to use only `ERC4626` vaults that are protected against share inflation.

- Slippage check on `withdraw` is not really useful. re-trying to withdraw later won't get you better conditions. so you effectively brick the `withdraw`.
- Slippage check on `deposit` could make sense yes at a first sight. But actually you can bypass it by doing a `deposit` - `withdraw` sequence.

5.1.3 Checks missing to make sure the exact number of assets withdrawn from deallocation

Severity: Medium Risk.

Context: [MorphoVaultV1Adapter.sol#L89](#), [VaultV2.sol#L612](#).

Description: When the adapter's deallocation endpoint is called, checks are missing to make sure the underlying token balance of the adapter gets increased by the same amount of `assets` that were provided to the `IERC4626(V).withdraw(assets, ...)`. In the parent call frame in the parent vault when the `deallocateInternal(...)` gets called, one has the following line:

```
SafeERC20Lib.safeTransferFrom(asset, adapter, address(this), assets);
```

The above line only guarantees that the `adapter`'s underlying token's balance after the `deallocate` call is at least `assets`. But this amount could be a sum of amount returned by the `ERC4626` yield source and some potential donated (or by some other means) underlying tokens that were already in the adapter.

Recommendation: Perform snapshot of the underlying token balance of the adapter before and after the call to `IERC4626(V).withdraw(assets, ...)` to make sure the change is at least `assets`.

Morpho: The integrated `ERC4626` vaults must transfer the right amount of assets on `withdraw()`.

Spearbit: Acknowledged.

5.1.4 Perform atomic approvals and add checks when depositing assets into the yield source

Severity: Medium Risk.

Context: [MorphoVaultV1Adapter.sol#L46](#), [MorphoVaultV1Adapter.sol#L71](#).

Description/Recommendation: When the current adapter gets used for a general `ERC4626` yield source, it would be best to perform atomic approvals when `allocate(...)` is called instead of giving an infinite approval to the yield source.

Moreover, perform underlying asset snapshots of the adapter balance and make sure the change is at most the `assets` provided.

The above would make sure any skim-able underlying amounts would not get transferred to the `ERC4626` yield source.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.2 Low Risk

5.2.1 Unsafe type casts might not be safe for a general `ERC4626` Vault

Severity: Low Risk.

Context: [MorphoVaultV1Adapter.sol#L77](#), [MorphoVaultV1Adapter.sol#L95](#).

Description: The unsafe `int256(...)` type casts in `MorphoVaultV1Adapter` might not be safe for a general `ERC4626` Vault (an overflow might go unnoticed).

Recommendation: The above should be documented for the general `ERC4626` Vault to make sure only vaults that would operate in the safe range of the cast (`type(int256).max` for return value of the `previewRedeem`) can only be used. Or the type conversions should be replaced by a safe cast, but that could open up a potential DoS attack vector if used along an incompatible yield source vault.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.2.2 Temporary DoS of a `VaultV2` by a general `ERC4626` yield source

Severity: Low Risk.

Context: `MorphoVaultV1Adapter.sol#L89-L91`, `MorphoVaultV1Adapter.sol#L112`, `VaultV2.sol#L431-L444`, `VaultV2.sol#L658-L660`.

Description: Upon interest accrual, a call is made to the `realAssets()` of all registered adapters. And if those adapters have allocations and are using an `ERC4626` yield source, the following call gets made to the underlying yield source vault:

```
IERC4626(V).previewRedeem(
    IERC4626(V).balanceOf(address(this))
)
```

If either of the `previewRedeem` or `balanceOf` of would revert for any reasons malicious or not many critical operations of the `VaultV2` (the parent vault of the adapter) would be blocked till that issue is resolved or the correspond adapter is removed from the registered adapter set (this operation is **time locked**).

i Note:

The above can also block deallocations from the yield source. This can also happen if the `IERC4626(V).withdraw(...)` would also revert.

Recommendation: The above scenario should at least be documented and monitored for malicious behaviours since it can affect all other yield sources that the `VaultV2` is exposed to through mix of allocations.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.2.3 Faulty `ERC4626` yield sources can gradually drain idle or adapter-allocated liquidity

Severity: Low Risk.

Context: `MorphoVaultV1Adapter.sol#L109-L115`, `VaultV2.sol#L656-L662`.

Description: If a faulty `ERC4626` reports incorrect and potentially inflated live allocations via `realAssets()`, allocations can be skewed over time, effectively pulling excess liquidity from:

- Idle liquidity.
- Liquidity allocated to the liquidity adapter.
- Other allocations via deallocation or force deallocation.

as time passes.

This means a misreporting `ERC4626` yield source can introduce risk to both idle funds and other allocations.

Recommendation: The owner, curator, and allocators should perform thorough due diligence on adapters and their underlying `ERC4626` yield sources to minimize this risk. Users should understand that their safety depends on those operators' selection and ongoing monitoring.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.2.4 Skim can't be used to withdraw rewards which require authentication

Severity: Low Risk.

Context: [MorphoVaultV1Adapter.sol#L49-L64](#).

Description: `skim` functionality is to collect airdropped `MORPHO` incentives/rewards. However, this is not a standard method of claiming extra incentives across all protocols. If the concerned vault's incentives and rewards claiming requires authentication such as `msg.sender` or some specific crafted call-data which is not possible from the current Morpho vault adapter, then the rewards would be lost.

some of the examples are following:

- Aave v3 rewards controller.
- Lido st vaults.
- Eigenlayer AVS.

(**note:** the above-mentioned examples are not directly `ERC4626` compliant contracts, and only cause issues when an `ERC4626` vault provides liquidity to them and doesn't sell and auto-harvest rewards to asset).

Recommendation: Document the compatibility for claiming incentives and rewards from contracts which do not have permissionless claiming of assets like Morpho's URD.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.2.5 Indirect shares donation to adapter may drift accounting

Severity: Low Risk.

Context: [MorphoVaultV1Adapter.sol#L109-L115](#).

Description: `MorphoVaultV1Adapter` doesn't have donation resistance (unlike market v1 adapter), so even if underlying vault is donation resistant, the shares of that vault could be directly transferred to increase allocation.

The supply caps don't prevent this, as it's not an allocation but rather a direct transfer, and will be clamped by `maxRate`.

Recommendation: `ERC4626` vault which is to be integrated should ensure there is no economic incentive to donate in following cases:

- Share donation when `allocation() == 0`, hides donation. This scenario could be potentially combined with `forceDeallocate`.
- Share donation when `allocation() != 0`, socializes but rate limited. this scenario could be potentially combined with `flashloan + deposit` before donating.

- Shares cannot be obtained cheaper than exchange rate of vault. especially, If a `VaultV2` has more than one adapter.

Also, any time-locked actions pending for execution should be considered when evaluating for donation resistance.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.3 Informational

5.3.1 `VaultV2` allocation to a general `ERC-4626` via `MorphoVaultV1Adapter`

Severity: Informational.

Context: [MorphoVaultV1Adapter.sol#L24](#).

Description: The `MorphoVaultV1Adapter` implicitly assumes **synchronous, reliable liquidity** and **stable, non-manipulable accounting views**. The following patterns/behaviours of a general `ERC4626` vault can break these assumptions:

A. Asynchronous withdrawals / request–claim flows: Forbidden: any vault where assets are not returned immediately on `withdraw`.

Red flags:

- Withdrawal queues.
- Redeem requests + claim steps.
- Cooldowns / unbonding periods.
- Epoch-based exits.
- ERC-7540-style async ERC-4626 behavior.

Why this breaks

`VaultV2` expects **atomic deallocation**. Queued exits can make `withdraw()` revert or defer settlement.

Examples of Vaults.

- [Mellow](#): withdrawal queue architecture.
- [Maple Finance](#): Withdrawal requests & processing delays.
- [ERC-7540 vaults](#).
- [OpenEden TBILL](#): Redemption queue + KYC.
- [Some Centrifuge vaults](#): async vault flows.

B. Liquidity-gated withdrawals (bank-run / utilization risk): Forbidden (with current adapter): vaults where `withdraw()` can fail due to insufficient liquidity.

Common sources:

- Lending protocol wrappers.
- Strategy vaults that deploy most capital.

Examples of Vaults.

- [Aave](#): withdrawals limited by available liquidity.
- [Compound v2](#): redemptions fail if insufficient cash.
- [Yearn v3](#): withdrawal limits based on strategy liquidity.

- [Sommelier](#).

Why this breaks

The adapter does **not** consult `maxWithdraw` before calling `withdraw`.

i Note:

This also applies to a `VaultV2` being used as a yield source.

C. Admin / guardian controls that can block exit: Forbidden or requires explicit trust acceptance.

- Pausable withdrawals.
- Emergency shutdown modes.
- Allowlists / KYC / permissioned redeem.
- Dynamic withdrawal caps that can drop to zero.

References.

- [OpenZeppelin ERC-4626 customization & security considerations](#).
- [Yearn v3](#): shutdown & withdraw-limit mechanisms.
- [OpenEden](#).
- RWA / private credit vaults: often NAV-based, permissioned, or async.

Why this breaks

VaultV2 can become permanently stuck.

D. `previewRedeem` is unreliable or manipulable: Forbidden: vaults where `previewRedeem` is:

- Manipulable (donation / inflation attacks).
- Dependent on spot prices or oracles.
- Dependent on state that can be adversarially perturbed.
- Revert-prone (stale NAV, paused oracle).

E. Non-standard underlying ERC-20 behavior: Forbidden.

- Fee-on-transfer tokens.
- Rebasing tokens (unless explicitly handled).
- Callback / hook tokens.

References.

- OpenZeppelin ERC-4626 token [compatibility notes](#).
- [Compound ERC-4626 wrapper](#) for rebasing tokens.

i Note:

The NatSpec comments on `VaultV2` assumes that the assets chosen do not have these behaviours.

F. Reentrancy during deposit / withdraw: Forbidden.

- Vaults or strategies that can re-enter VaultV2 or the adapter.
- Callback tokens or untrusted external calls during hooks.

i Note:

The NatSpec comments on `VaultV2` assumes that the assets chosen do not have these behaviours.

G. View-function liveness (DoS-by-view): Forbidden.

- `previewRedeem`, `totalAssets`, or related views can revert.
- Views depend on stateful accruals that can fail.
- Views are gas-heavy enough to cause DoS.

Why this breaks

VaultV2 accounting may rely on these views; revert = protocol-level DoS.

H. Circular/Looped Strategies: Scenario where down the line some sub-yield source of the `ERC-4626` deposits into the parent vault perhaps should be avoided or analysed meticulously (at least as a toy model):

$$V_p \rightarrow E_{4626} \rightarrow V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V_n \rightarrow V_p$$

Or in general one might like to have the graph of fund allocation to yield sources to not have a loop, aka it should be a tree.

Other Examples of Vaults: Smaller list of other vaults to investigate:

- [Spark sDAI](#).
- [Frax sFRAX / sfrxETH](#).
- [Aura ERC-4626 reward pools](#).
- [Beefy ERC-4626 wrappers](#).

Recommendation: The above assumptions and features that would need to be avoided should be documented.

Morpho:

B. Liquidity-gated withdrawals (bank-run / utilization risk): This is a bit too restrictive. e.g. vault-v2 itself would break with this requirement. The important property though is that the vault should be non-custodial, in the sense that we use it in Morpho vault-v1 and vault-v2 (you can always exit from the vault, either because it's liquid either thanks to IKR, before the curator can change the allocation in a way you disagree with).

5.3.2 Potential discrepancies between `previewRedeem`, `redeem` and `withdraw`

Severity: Informational.

Context: [MorphoVaultV1Adapter.sol#L73](#).

Description: Based on EIP-4626 specification, in general we have:

```
previewRedeem(s) <= redeem(s, ...)
```

Thus the fetched allocation snapshots (or their changes) through `allocate`, `deallocate` and `realAssets()` of a general ERC4626 adapter are underestimation of the actual potential allocation that can be withdrawn (this is if the ERC4626 yield source vault would conform fully to the specification).

Recommendation: This potential discrepancy can be documented.

Morpho: Acknowledged.

Spearbit: Acknowledged.